

Efficient Bilevel Optimization for CKA-Guided MoE Upcycling

Zhiyuan Yu^{1,2} Enneng Yang¹ Hao Jiang³ Guojie Zhu³ Feihong He¹ Peng Wang¹ Li Shen^{1,2}

Abstract

Upcycling, a strategy that initializes Mixture-of-Experts (MoE) by replicating pre-trained feed-forward or MoE networks to expand model capacity, has become a popular method in continual learning due to its effectiveness in mitigating catastrophic forgetting. However, existing paradigms indiscriminately expand capacity to prioritize performance at the cost of severe inefficiency, introducing severe parameter redundancy and failing to exploit structural heterogeneity. To address this, we investigate the determinants of forgetting in training dynamics using Centered Kernel Alignment (CKA) and loss landscape flatness to analyze the behavior of pre- and post-expansion MoE layers, uncovering instability in deep-layer representations and heterogeneous expert sensitivity to new tasks, thereby demonstrating the potential of selective upcycling to eliminate redundancy. Consequently, we propose a dynamic bilevel optimization framework to guide adaptive upcycling, featuring an outer loop employing a Gumbel-Softmax differentiable mask to perform Neural Architecture Search (NAS) for adaptive growth, while an inner loop optimizes weight updates via task objectives and CKA-regularized replay. Experiments on the TRACE benchmark demonstrate that our proposed method achieves better average accuracy with 80% forgetting reduction, while effectively eliminating 60% of redundant parameter expansion that standard upcycling would introduce.

1. Introduction

Large Foundation Models (Brown et al., 2020; Touvron et al., 2023) have emerged as the cornerstones of deep learn-

¹School of Cyber Science and Technology, Shenzhen Campus of Sun Yat-sen University, China ²Shenzhen Loop Area Institute, China ³Huawei, China. Correspondence to: Li Shen <shenli6@mail.sysu.edu.cn>.

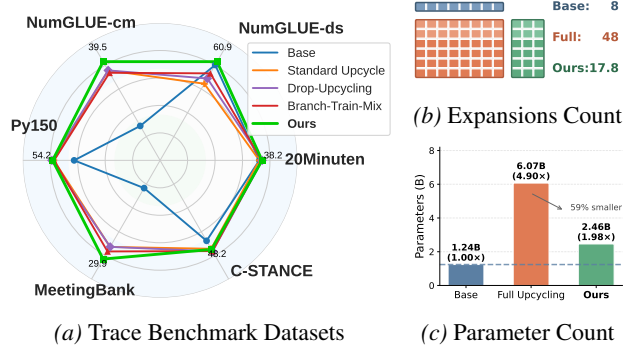


Figure 1. **Efficiency Overview.** (a) We evaluate on 6 diverse datasets from TRACE. (b) Our method expands significantly fewer experts than standard upcycling. (c) Consequently, our method achieves 60% parameter reduction compared to full upcycling, significantly improving parameter efficiency.

ing, demonstrating exceptional performance across diverse tasks. The Mixture-of-Experts (MoE) architecture (Shazeer et al., 2017; Jiang et al., 2024; Team et al., 2024) further enhances this capability through multiple specialized experts and scaling up. However, a model’s knowledge becomes static upon pre-training completion. Introducing new information requires continual training on sequential tasks, raising the critical challenge of catastrophic forgetting (McCloskey & Cohen, 1989; French, 1999), wherein new knowledge overwrites old knowledge and causes the model to lose previously acquired capabilities.

Continual learning (CL) methods have emerged to mitigate forgetting through various paradigms (Wang et al., 2024). Among architecture-based CL approaches, dynamic strategies such as Upcycling have shown notable promise for scaling large models (Komatsuzaki et al., 2022; Sukhbaatar et al., 2024; Liao et al., 2025; Nakamura et al., 2025). Upcycling transforms dense checkpoints into sparse MoE architectures or scales existing MoEs, creating dedicated parameter subspaces for new tasks while preserving previous knowledge. However, typical upcycling and model-growth paradigms rely on indiscriminate expansion, for example, brute-force splitting of dense layers into experts or doubling experts across all layers when learning new tasks to expand capacity. While effective for capacity, such heuristic strategies prioritize performance at a steep cost to efficiency, leading to significant parameter redundancy (see Section 3). This raises a critical research question: *How can we identify*

and validate metrics that guide whether, where, and how to expand MoE networks?

To address this question, we investigate forgetting through the lens of the **stability-plasticity dilemma**, a principle rooted in neuroscience that captures the trade-off between retaining old knowledge and acquiring new information, and has become central to continual learning (Parisi et al., 2019). Our diagnostic study (see Section 3) combines Centered Kernel Alignment (CKA) (Kornblith et al., 2019) with Hessian-based flatness analysis to identify where forgetting occurs and where expansion is needed. Results show that shallow and middle layers retain high representational stability, whereas deep layers exhibit substantial drift and require expansion. We observe expert-specific heterogeneity indicating that different experts require different expansion strategies rather than uniform treatment.

These insights motivate our NAS-Guided Upcycling framework design. Unlike prior work that employs predetermined expansion schedules, we formulate expert expansion as a learnable architectural decision through differentiable neural architecture search within a supernet architecture. We cast this as a bilevel optimization problem: the inner loop optimizes expert parameters with CKA-based regularization to preserve representational stability, while the outer loop learns selective expansion policies via Gumbel-Softmax masks that balance knowledge preservation, capacity acquisition, and parameter efficiency.

Empirical evaluations on the TRACE benchmark (Wang et al., 2023b) using Llama-3.2-1B-Instruct validate the effectiveness of our method. By dynamically allocating capacity, our method achieves an effective expansion rate of approximately 38.7%, discarding 60% of redundant parameters introduced by standard upcycling baselines. Our method achieves 45.05% last accuracy, surpassing other upcycling baselines in both accuracy and forgetting, with an 80% reduction in forgetting compared to sequential fine-tuning and a 60% reduction compared to standard upcycling.

Our contributions are threefold:

- We introduce a diagnostic framework integrating CKA and Hessian-based flatness analysis that reveals critical heterogeneity across layers and experts, providing insights for selective expansion strategies.
- We propose a novel upcycling algorithm that takes expert expansion as a learnable decision via Gumbel-Softmax sampling within an interleaved bilevel optimization framework, utilizing CKA dynamics to guide optimal trade-offs between plasticity and stability.
- We implement and evaluate our method on the TRACE benchmark, demonstrating higher accuracy with lower

forgetting metrics while eliminating redundant parameters compared to existing CL and upcycling baselines.

2. Related Work

Continual Learning. Continual learning methods fall into four paradigms (Wang et al., 2024). **Memory-based** methods mitigate forgetting by retaining historical distributions through experience replay (Chaudhry et al., 2019) or generative models. **Regularization-based** methods preserve knowledge by constraining plasticity, e.g., penalties on important parameters (EWC (Kirkpatrick et al., 2017)), knowledge distillation (LwF (Li & Hoiem, 2017)), momentum-filtered selective updates (MoFO (Chen et al., 2024)). **Subspace-based** methods minimize interference by restricting optimization to orthogonal gradient (Farajtabar et al., 2020) or feature subspaces (e.g., O-LoRA (Wang et al., 2023a)). **Architecture-based** methods isolate task knowledge via structural design: either within fixed capacity (Mallya & Lazebnik, 2018) or through dynamic expansion (depth- or width-wise model growth (Rusu et al., 2016)). Our work belongs to the architecture-based line. We use replay and regularization as in TRACE (Wang et al., 2023b); our core contribution is a bilevel framework that learns *where* to expand the MoE, with ablations showing gains beyond replay or regularization alone.

MoE Growth and Upcycling. Upcycling recycles pre-trained dense weights into MoE structure to add capacity; it has become a central architectural strategy for continual learning. *Sparse Upcycling* (Komatsuzaki et al., 2022) established the baseline for dense-to-MoE transformation. *Branch-Train-MiX* (Sukhbaatar et al., 2024) introduced parallel expert training for scalable MoE construction. *Innovator* (Liao et al., 2025) showed fine-grained upcycling (splitting FFN dimensions) for continued pre-training; *Drop-Upcycling* (Nakamura et al., 2025) randomly selects and reinitializes experts to improve diversity. For continual learning specifically, *Lifelong-MoE* (Chen et al., 2023) uniformly expands experts across all MoE layers per new task, preserving knowledge via distillation from old experts and gating. *LLaVA-CMoE* (Zhao et al., 2025) uses probe-guided signals to decide when and where to add experts in vision-language settings. These methods either expand uniformly or rely on limited diagnostics, without explicitly balancing stability vs. plasticity or supporting fine-grained, layer- and expert-wise selectivity. In contrast, we use differentiable NAS in a bilevel framework to learn selective expansion, with CKA and flatness guiding redundancy reduction while preserving performance (Appendix E).

3. Analyzing Upcycling Dynamics

To understand why indiscriminate upcycling is inefficient, we conduct a diagnostic study on a Llama-3.2-1B model

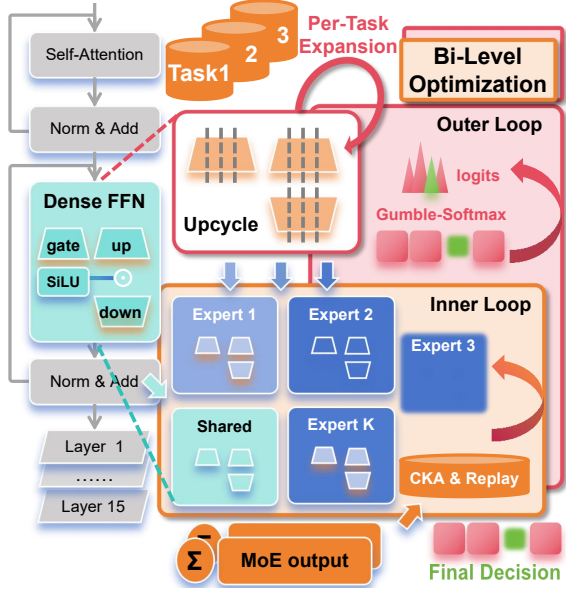


Figure 2. Bi-Level framework. *Outer loop*: differentiable NAS via Gumbel-Softmax masks to expand or recycle experts per task. *Inner loop*: expert weights optimized with CKA and replay for stable continual learning.

firstly upcycled to an 8-expert MoE and trained on a sequence of 6 diverse tasks from the TRACE benchmark, jointly analyzing CKA-based representational stability and loss landscape flatness as proxies for plasticity. In the standard upcycling paradigm, each new task triggers an upcycle operation: dense FFN layers are divided to create 8 new experts that are added to the existing expert pool, and the routing mechanism is expanded accordingly to accommodate the growing number of experts. Consistent with our formulation in Section 1, this study reveals marked heterogeneity across layers and experts in their representational preservation and adaptation needs, which in turn motivates our bilevel Differentiable NAS-Guided Upcycling framework for selective, parameter-efficient expansion.

3.1. Preliminaries

Continual Learning Formulation. We consider a sequential learning setup where a model is trained on a sequence of tasks $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$. Each task T_t arrives sequentially with its own training dataset $\mathcal{D}_t$. The goal is to train the model on the current task T_t to minimize the loss $\mathcal{L}_t$, while maintaining performance on previous tasks $T_{1:t-1}$ and preserving the model’s general capabilities.

Upcycling via Weight Slicing. Figure 2 illustrates our proposed bi-level framework with CKA- and flatness-guided optimization over the standard upcycling method. The standard upcycling paradigm adopted in prior work (Komat-suzaki et al., 2022; Chen et al., 2023; Liao et al., 2025) can be summarized as follows. Upcycling transforms a dense

Feed-Forward Network (FFN) into a Mixture-of-Experts (MoE) layer to increase capacity or modularity without significantly increasing inference cost. A standard dense FFN layer in Transformer models (e.g., LLaMA (Touvron et al., 2023)) is defined as:

$$\text{FFN}(x) = W_{\text{down}}(\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x), \quad (1)$$

where $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{H \times d}$ and $W_{\text{down}} \in \mathbb{R}^{d \times H}$, with d and H being the hidden and intermediate dimensions.

In the standard upcycling approach adopted in our work, we initialize the experts by slicing the pretrained weights of the dense FFN. Specifically, for a target of K experts, we partition the intermediate dimension H into K equal parts of size $h = H/K$. The weights for the i -th expert E_i are initialized as:

$$W_{\text{gate}}^{(i)} = W_{\text{gate}}[(i-1)h : ih, :], \quad (2)$$

$$W_{\text{up}}^{(i)} = W_{\text{up}}[(i-1)h : ih, :], \quad (3)$$

$$W_{\text{down}}^{(i)} = W_{\text{down}}[:, (i-1)h : ih], \quad (4)$$

where we use Python-style slicing notation. The MoE layer output is then computed as:

$$\text{MoE}(x) = \sum_{i=1}^K g(x)_i E_i(x), \quad (5)$$

where $g(x)$ is the routing function (e.g., Softmax) and $E_i(x) = W_{\text{down}}^{(i)}(\sigma(W_{\text{gate}}^{(i)}x) \odot W_{\text{up}}^{(i)}x)$. In the standard upcycling approach, when a new task arrives, each dense FFN layer undergoes the same weight slicing procedure to create 8 new experts that are added to the existing expert pool, and the routing function is expanded accordingly to accommodate the growing number of experts. While this approach improves performance over dense baselines (see Figure 3, left), it expands capacity indiscriminately across all layers, leading to parameter inefficiency.

3.2. Training Dynamics with CKA and Flatness

To identify where expansion is redundant and where additional capacity is beneficial, we analyze training dynamics and expert usage to determine, per layer, whether expansion is unnecessary or needed.

Standard Upcycling is parameter-inefficient. As shown in the left panel of Figure 3, Standard Upcycling achieves higher average performance than the dense baseline (41.21% vs 31.09%) and substantially mitigates forgetting (backward transfer: -9.05 vs -20.13), confirming that additional capacity is beneficial. However, this naïve strategy increases the parameter count by $4.9\times$ relative to the base model (6.07B vs 1.24B). By contrast, our selective upcycling attains comparable performance with only $1.98\times$ parameters (2.46B), corresponding to a 60% reduction in model size

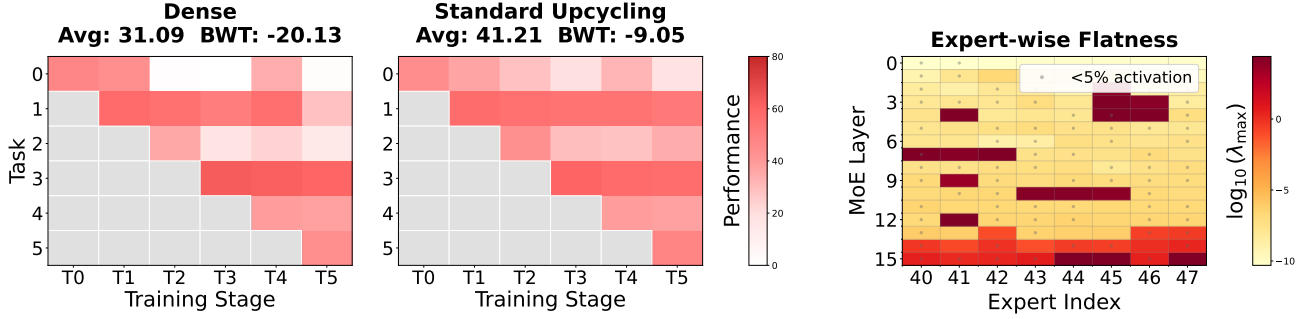


Figure 3. Empirical analysis of Standard Upcycling on Llama-3.2-1B. Left: Dense vs Standard Upcycling performance heatmaps. Right: expert-wise flatness $\log_{10}(\lambda_{\max})$; gray dots mark rarely-activated experts.

(Figure 1c). The right panel of Figure 3 visualizes expert flatness and routing frequency across all MoE layers and experts after training on T_5 : lighter colors denote flatter experts, and gray dots mark experts that are activated on fewer than 5% of tokens. A large fraction of the newly added experts, particularly in shallow and middle layers, fall into this low-usage regime, indicating that indiscriminate expansion creates many under-utilized experts that contribute little to new-task learning.

3.2.1. LAYER-WISE REPRESENTATION SIMILARITY

To study how expansion changes internal representations, we employ *linear* Centered Kernel Alignment (CKA) (Kornblith et al., 2019) as a layer-wise similarity measure. For two activation matrices $X, Y \in \mathbb{R}^{n \times d}$ (rows are samples, columns are centered features), linear CKA is defined as

$$\text{CKA}(X, Y) = \frac{\|Y^\top X\|_F^2}{\|X^\top X\|_F \|Y^\top Y\|_F}. \quad (6)$$

Specifically, we compare the MoE layer’s output on *old-task data* ($\mathcal{D}_{\text{old}}$) before and after expansion. Let H_ℓ be the layer output before expansion and H_ℓ^{up} be the output after expansion and training on new tasks. We then compute a per-layer similarity score

$$\text{CKA}_\ell = \text{CKA}(H_\ell(\mathcal{D}_{\text{old}}), H_\ell^{\text{up}}(\mathcal{D}_{\text{old}})), \quad (7)$$

using a small random subset of old-task activations and focusing on deeper layers where representational drift is most significant. High CKA values indicate that the layer’s functional behavior on old data is preserved, suggesting that the existing capacity was sufficient and the expansion might have been redundant. Figure 4(a) shows CKA similarity on old tasks T_0 – T_4 (horizontal axis) after the model has been trained on T_5 : shallow and middle layers maintain CKA on most tasks while deep layers drop, indicating that deeper representations drift more strongly as they adapt to encode task-specific features.

Deep layers exhibit substantial representational drift and expansion needs. As shown in Figure 4(a), CKA on

old tasks T_0 – T_4 (after training on T_5) varies by layer: the deepest layers (e.g., Layer 15) exhibit substantially lower CKA on these tasks, while shallow and middle layers retain higher CKA. This pattern suggests that early layers encode more task-agnostic features that generalize across tasks, while later layers specialize and are therefore more sensitive to upcycling. In line with this, the expansion decisions produced by our method during dynamic training vary substantially by depth: e.g., Layer 15 (deep) receives 87.5% expert expansion, whereas Layer 12 (middle) receives only 37.5%, reflecting that deep layers demand substantially more new capacity whereas middle layers can often recycle existing experts without sacrificing performance. Figure 4(b) further shows that flatness of the latest expert group varies by layer and by expert; expansion decisions thus benefit from considering both layer depth and expert-specific characteristics.

3.2.2. LOSS LANDSCAPE FLATNESS

Complementary to CKA, which measures changes in representations on old tasks, we also investigate the loss landscape geometry (Foret et al.) to characterize where the model is most sensitive to new-task updates. Concretely, we empirically compare several standard flatness measures, such as gradient norm, random-direction landscape sharpness, Fisher trace, and the top Hessian eigenvalue, and verify that they induce highly consistent layer-wise rankings (see Appendix B, Figure 6). Motivated by this agreement, we adopt a gradient-norm based proxy in our main algorithm and figures due to its much lower computational cost. Using this proxy, for layer parameters θ_ℓ and loss $\mathcal{L}$ on *new-task* data, we define $\text{Flatness}_\ell = \frac{1}{|\theta_\ell|} \|\nabla_{\theta_\ell} \mathcal{L}\|_2$, computed per expert and per layer using mini-batches of new-task examples. For flatness visualizations in our figures we use the top Hessian eigenvalue: with $H_\ell = \nabla_{\theta_\ell}^2 \mathcal{L}$ denoting the Hessian of $\mathcal{L}$ with respect to θ_ℓ , $\lambda_{\max} = \lambda_{\max}(H_\ell)$. Alternative proxies yield qualitatively similar layer-wise rankings and are discussed in more detail in Appendix B. Figure 4(b) visualizes flatness at checkpoint 5 (i.e., after training up to task 5), using $\lambda_{\max}$: the horizontal axis is the latest expanded expert group (experts 1–8), and the vertical axis

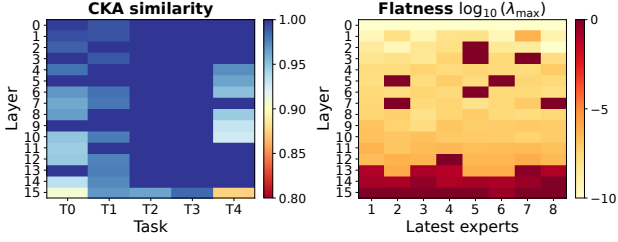


Figure 4. **Upcycling Dynamics.** (a) CKA similarity between the dense and upcycled models on old tasks T_0 – T_4 after training on T_5 , shown across layers. Layer 15 (deep) exhibits substantially lower CKA values, indicating strong representational drift on old data. (b) Flatness of the latest expanded expert group measured by $\log_{10}(\lambda_{\max})$ for each layer–expert pair at checkpoint 5; larger values correspond to greater sensitivity to task-specific adaptation.

is layer; higher $\log_{10}(\lambda_{\max})$ indicates sharper minima and greater sensitivity to new-task adaptation.

Flat regions resist both forgetting and new learning. Layers with lower flatness are more robust to catastrophic forgetting but also less receptive to new knowledge; expanding experts in these regions contributes minimally to the model’s effective capacity for new tasks.

Flatness aligns with CKA trends across layers. Shallow and middle layers exhibiting high CKA similarity (low drift) on old tasks also tend to have flatter loss landscapes, suggesting that these layers encode task-agnostic features that generalize across tasks. Conversely, deep layers with lower CKA show sharper loss landscapes (Figure 4(b)), indicating sensitivity to task-specific adaptation. Deeper layers exhibit consistently sharper loss landscapes compared to shallow layers, aligning with our CKA findings that deeper layers require more task-specific adaptation.

3.3. Implications for Selective Upcycling

Taken together, these diagnostics show that CKA and flatness effectively characterize where forgetting occurs and where new capacity is needed, and can thus guide expansion decisions. Deep layers both drift more (low CKA) and sit in sharper regions of the loss landscape, making them natural candidates for additional experts. Moreover, expert-wise flatness and routing statistics reveal that only a subset of experts within these layers truly benefit from added capacity. In contrast, shallow and middle layers, along with many of their experts, remain relatively stable and flat, indicating that much of their existing capacity can be safely recycled. This pattern indicates that indiscriminate expansion fails to distinguish between layers that require new capacity and those that already possess sufficient generalizable knowledge, suggesting a substantial opportunity for optimization if we can properly measure the necessity of expansion. Rather than uniformly doubling expert counts across all layers, our proposed method uses these observations to guide a learnable expansion policy that focuses new capacity where it is most

needed while avoiding redundant growth elsewhere.

4. Methodology

Building on the analysis in Section 3, we now turn these diagnostic insights into an actionable training algorithm. We introduce a Differentiable NAS-Guided Upcycling framework that casts expert expansion as a bilevel optimization problem (Franceschi et al., 2018). Instead of expanding all experts uniformly, our method treats *whether and where to expand* as a learnable architectural decision, guided by the layer-wise CKA and flatness patterns observed in our study, and is solved in practice with a first-order alternating scheme that interleaves expert gradient steps with periodic mask updates.

Bilevel Formulation. Our proposed method formulates continual learning as a bilevel optimization problem:

$$\begin{cases} \text{Outer (Mask):} & \min_{\pi} \mathcal{L}_{\text{mask}}(\theta^*, \pi), \\ \text{Inner (Expert):} & \theta^* = \arg \min_{\theta} \mathcal{L}_{\text{expert}}(\theta, \pi), \end{cases} \quad (8)$$

where the inner loop updates model parameters under CKA and replay regularization, and the outer loop updates a set of binary-like masks that decide whether to expand or recycle for each expert. Under standard assumptions (unbiased gradient, bounded variance, smoothness), the outer-loop iterates satisfy a convergence bound on the average expected squared gradient norm; see Appendix G. We summarize the overall training procedure in Algorithm 1. Next, we give a detailed description of our method.

4.1. Inner Loop: Expert Training

In the inner loop, we treat the architecture mask as fixed and update only the parameters of the trainable experts. The objective combines three terms: a supervised loss on the current task, a CKA-based penalty that discourages deep-layer representations from drifting too far from the pre-upcycling model, and a replay loss that maintains performance on previously seen tasks. Formally, the inner-loop objective is

$$\mathcal{L}_{\text{expert}} = \mathcal{L}_{\text{task}} + \lambda_{\text{CKA}} \cdot \mathcal{L}_{\text{CKA}} + \lambda_{\text{replay}} \cdot \mathcal{L}_{\text{replay}}, \quad (9)$$

where $\mathcal{L}_{\text{task}}$ is the task loss on current data; $\mathcal{L}_{\text{CKA}}$ is the mean over deeper layers of $1 - \text{CKA}(\Phi_{\ell}, \Phi_{\ell}^{\text{old}})$ on a mini-batch of previous activations, penalizing representational drift; $\mathcal{L}_{\text{replay}}$ is task loss evaluated on a mini-batch from the replay buffer $\mathcal{R}$. The coefficients λ_{CKA} and λ_{replay} are tuned on a held-out split and used as priors (see appendix).

4.2. Outer Loop: Architecture Mask Optimization

Supernet and Mask-based NAS. For each task $t \geq 1$, the outer loop first constructs a *supernet* that exposes both old and new experts to the mask optimizer. For every existing

Algorithm 1 Bilevel Optimization for Selective MoE Upcycling

Require: Tasks $\{\mathcal{T}_t\}_{t=0}^T$, base model θ_{base} , target expansion rate r_{target}

Ensure: Continually learned MoE model θ

- 1: **Initialization (Task 0):**
- 2: Convert θ_{base} to MoE θ , train on $\mathcal{T}_0$, init Replay Buffer $\mathcal{R}$
- 3: **Continual Learning Loop (Task $t \geq 1$):**
- 4: **for** $t = 1$ to T **do**
- 5: **Phase 1: Supernet Setup**
- 6: Clone experts: $e_{\ell,k}^{\text{old}}$ (frozen), $e_{\ell,k}^{\text{new}}$ (trainable)
- 7: Initialize mask logits α
- 8: **Phase 2: Interleaved Bilevel Optimization**
- 9: **for** step = 1 to total_steps **do**
- 10: *Inner Loop: Expert Training*
- 11: $\pi \leftarrow \text{Gumbel-Softmax}(\alpha, \tau)$; use π for expert routing
- 12: $\mathcal{L}_{\text{expert}} \leftarrow \mathcal{L}_{\text{task}} + \lambda_{\text{CKA}} \mathcal{L}_{\text{CKA}} + \lambda_{\text{replay}} \mathcal{L}_{\text{replay}}$
- 13: Update $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{expert}}$
- 14: *Outer Loop: Mask Learning (every N steps)*
- 15: **if** step % $N = 0$ **then**
- 16: $\pi \leftarrow \text{Gumbel-Softmax}(\alpha, \tau)$
- 17: $\bar{\pi} \leftarrow \frac{1}{LE} \sum_{\ell,k} \pi_{\ell,k}$
- 18: $\mathcal{G}_{\text{knowledge}} \leftarrow (\mathcal{L}_{\text{recycle}} - \mathcal{L}_{\text{expand}}) \cdot \bar{\pi}$
- 19: $\mathcal{L}_{\text{mask}} \leftarrow \mathcal{L}_{\text{replay}}(\pi) - \gamma \mathcal{G}_{\text{knowledge}} + \beta \mathcal{L}_{\text{balance}}(\pi)$
- 20: Update $\alpha \leftarrow \alpha - \eta_{\alpha} \nabla_{\alpha} \mathcal{L}_{\text{mask}}$
- 21: $\tau \leftarrow \max(\tau_{\text{final}}, \tau \cdot \rho)$
- 22: **end if**
- 23: **end for**
- 24: **Phase 3: Architecture Finalization**
- 25: Keep $e_{\ell,k}^{\text{new}}$ if $\sigma(\alpha_{\ell,k}) > 0.5$, else keep $e_{\ell,k}^{\text{old}}$
- 26: $\mathcal{R} \leftarrow \mathcal{R} \cup \text{Sample}(\mathcal{D}_t)$
- 27: **end for**
- 28: **return** θ

expert $e_{\ell,k}^{(t-1)}$ at layer ℓ , we create a frozen copy $e_{\ell,k}^{\text{old}}$ and a trainable clone $e_{\ell,k}^{\text{new}}$. The output of expert (ℓ, k) in the supernet is a convex combination of the two:

$$o_{\ell,k}(x) = \pi_{\ell,k} e_{\ell,k}^{\text{new}}(x) + (1 - \pi_{\ell,k}) e_{\ell,k}^{\text{old}}(x), \quad (10)$$

where $\pi_{\ell,k} \in [0, 1]$ is a learnable mask applied inside the expert block. The standard MoE router still selects which experts to activate based on the input x , and the mask $\pi_{\ell,k}$ then determines whether an activated expert behaves as adapted or frozen. When $\pi_{\ell,k} = 1$, the new expert is effectively used (*expand*), whereas when $\pi_{\ell,k} = 0$, the old expert is retained (*recycle*). This separation keeps router training and load balancing stable, since they operate on expert indices while the mask only changes expert functions.

Mask Parameterization. We parameterize masks via logits $\alpha \in \mathbb{R}^{L \times E \times 2}$ for L layers and E experts per layer, and obtain masks via Gumbel-Softmax (Jang et al., 2016):

$$\pi_{\ell,k} = \text{Gumbel-Softmax}(\alpha_{\ell,k}, \tau)_1, \quad (11)$$

where τ is the temperature and the subscript selects the expansion probability. We employ the straight-through estimator so that the forward pass can use nearly discrete masks while gradients are propagated through the logits.

Balanced Mask Objective. The outer-loop loss balances replay performance, knowledge gain from expansion, and a soft target expansion rate:

$$\mathcal{L}_{\text{mask}} = \mathcal{L}_{\text{replay}}(\pi) - \gamma \mathcal{G}_{\text{knowledge}}(\pi) + \beta \mathcal{L}_{\text{balance}}(\pi), \quad (12)$$

where $\gamma \geq 0$ is the weight for the knowledge-gain term and $\beta \geq 0$ is the balance weight that scales the expansion-rate penalty (see Table 10 for default values). Further,

$$\mathcal{G}_{\text{knowledge}} = (\mathcal{L}_{\text{recycle}} - \mathcal{L}_{\text{expand}}) \cdot \bar{\pi}, \quad (13)$$

is the *knowledge gain* term that encourages expansion when new experts outperform old ones. Here, $\mathcal{L}_{\text{recycle}}$ is the loss with masks forced to 0 (strictly old experts), $\mathcal{L}_{\text{expand}}$ is the loss with masks forced to 1 (strictly new experts), and $\bar{\pi} = \frac{1}{LE} \sum_{\ell,k} \pi_{\ell,k}$ is the mean expansion rate. Let σ denotes the sigmoid function $\sigma(z) = 1/(1 + e^{-z})$. The balance term softly enforces a target expansion rate r_{target} with asymmetric penalties:

$$\mathcal{L}_{\text{balance}} = \begin{cases} 2\beta(r_{\text{target}} - \bar{\pi}) & \text{if } \bar{\pi} < r_{\text{target}}, \\ \sigma(\bar{\pi} - r_{\text{target}}), & \text{otherwise.} \end{cases} \quad (14)$$

Temperature Annealing. The Gumbel-Softmax temperature decays exponentially from soft to hard decisions,

$$\tau_t = \max(\tau_{\text{final}}, \tau_0 \cdot \rho^t), \quad (15)$$

where τ_0 and τ_{final} bound the temperature schedule and $\rho \in (0, 1)$ sets its decay rate.

Final decisions. After training task t , we finalize architectural choices by thresholding the learned logits:

$$\text{Decision}_{\ell,k} = \begin{cases} \text{expand} & \text{if } \sigma(\alpha_{\ell,k}) > 0.5, \\ \text{recycle} & \text{otherwise.} \end{cases} \quad (16)$$

The *expand* decision retains the newly trained expert and discards the frozen copy, while the *recycle* decision keeps the original expert (unfrozen for future adaptation) and discards the new clone.

4.3. Complexity and Efficiency Analysis

Complexity and efficiency, including memory footprint, computational cost (FLOPs), and convergence of the bilevel formulation, are central to assessing our method. Because only a subset of layers carry candidate experts and the doubled-expert state is transient (released after mask finalization), our training overhead remains close to Standard Upcycling while the deployed model is markedly smaller and faster at inference. We defer the full quantitative comparison against all baselines to Appendix D.

5. Experiments

5.1. Experimental Setup

Benchmark. We evaluate our method on the **TRACE** benchmark (Wang et al., 2023b), a multi-task sequential learning suite tailored for continual learning with large language models. Following the official protocol, we consider a sequence of six heterogeneous tasks covering various domains: MeetingBank (meeting summarization), Py150 (code completion), NumGLUE-cm (numerical common-sense), NumGLUE-ds (numerical reasoning), 20Minuten (text simplification), and C-STANCE (stance detection). This mixture of generation and classification tasks stresses both the model’s general capabilities and its robustness to catastrophic forgetting.

Backbones and Upcycling Configuration. Unless otherwise specified, our main results are reported on Llama-3.2-1B-Instruct upcycled to an 8-expert MoE using the weight-slicing strategy described in Section 3. To examine robustness across scales and architectures, we additionally evaluate on Llama-3.2-3B. All models are trained with DeepSpeed ZeRO (Rasley et al., 2020), using the same number of epochs (5 per task) and the same replay buffer budget across methods to ensure a fair comparison.

Baselines. We compare against strong continual learning and upcycling baselines, including **SeqFT**, **LoRA** (Hu et al., 2022), **O-LoRA** (Wang et al., 2023a), **EWC** (Kirkpatrick et al., 2017), **Replay**, **MoFO** (Chen et al., 2024), and **Standard Upcycle** (full upcycling). We also report an **Individual FT** upper bound, which fine-tunes a separate model per task and is therefore not a CL method.

5.2. Evaluation Metrics

For each task, we follow TRACE (Wang et al., 2023b) and adopt the standard task-specific metric: ROUGE-L for MeetingBank, code similarity for Py150, accuracy for NumGLUE-cm/ds and C-STANCE, and SARI for 20Minuten. To summarize performance over the entire task sequence, we report three standard continual learning metrics: *Last Accuracy* A_{last} (mean performance after learning all tasks), *Stage Accuracy* $\bar{A}$ (average of per-stage accuracies A_b), and *Backward Transfer* (BWT), which measures forgetting on past tasks. Let $A_{b,j}$ denote the performance on task j after training on task b among N tasks:

$$\begin{cases} A_{\text{last}} = \frac{1}{N} \sum_{j=1}^N A_{N,j}, \\ A_b = \frac{1}{b} \sum_{j=1}^b A_{b,j}, \quad \bar{A} = \frac{1}{N} \sum_{b=1}^N A_b, \\ \text{BWT} = \frac{1}{N-1} \sum_{j=1}^{N-1} (A_{N,j} - A_{j,j}), \end{cases} \quad (17)$$

Table 1. Comparison of CL methods on Llama3.2-1B. We report Last Acc, Stage Acc, and BWT. Bold numbers denote the best method. Results on Llama3.2-3B are in Appendix C. Note that Individual FT is not a CL method and is for reference only.

Methods\Metrics	Last Acc	Stage Acc	BWT
Individual FT	49.58	50.31	–
SeqFT	29.85±2.73	39.80±0.30	-21.46±3.69
LoRA	26.34±0.23	28.55±0.10	-20.58±0.20
EWC	35.97±0.68	42.69±0.05	-14.11±0.33
Replay	43.76±0.51	46.33±0.12	-4.93±1.03
MoFO	41.81±0.38	43.61±0.23	-7.80±0.21
Standard Upcycle	41.49±0.28	43.00±0.56	-8.64±0.90
Drop-Upcycle	41.43±0.78	43.79±0.13	-8.32±0.93
BTM	42.50±0.33	44.66±0.64	-7.32±0.19
Upcycling (Ours)	45.05±0.15	46.20±0.12	-4.05±0.18

where more negative BWT indicate larger degradation.

5.3. Main Results on TRACE

Table 1 reports the performance of all methods on Llama3.2-1B in terms of Last Acc, Stage Acc, and BWT; results on Llama3.2-3B are in Appendix C. On Llama3.2-1B, our method achieves the **highest Last Acc** (45.05%) among all continual learning baselines and the **best (least negative) BWT** (-4.05), indicating both the strongest final accuracy and the best anti-forgetting effect. Our bilevel upcycling framework consistently attains the best or near-best accuracy while substantially reducing forgetting. Figure 5 shows performance heatmaps for all compared methods on Llama-3.2-1B. Qualitatively, these results indicate that learning where to expand experts is more effective than indiscriminate full expansion: the model can allocate additional capacity only to layers and tasks that truly benefit from it, while preserving the representational subspaces that already generalize well across tasks.

One of the main benefits of our approach is parameter efficiency. Rather than expanding all layers uniformly, our method learns a selective expansion pattern based on layer-wise stability analysis. As shown in Figure 1c, our method achieves a final model size of 2.46B parameters ($1.98\times$ the base model), compared to 6.07B for full upcycling ($4.90\times$), representing a **60% parameter reduction** while matching or exceeding performance.

We further analyze where expansion occurs and how performance evolves. Layer-wise statistics show that deep, task-specific layers tend to receive more new experts, whereas middle layers are frequently recycled, corroborating the diagnostic patterns identified in Section 3. Temporal heatmaps of task performance across stages (Figure 5) reveal that earlier tasks retain strong performance even after later tasks are introduced, indicating mitigated catastrophic forgetting.

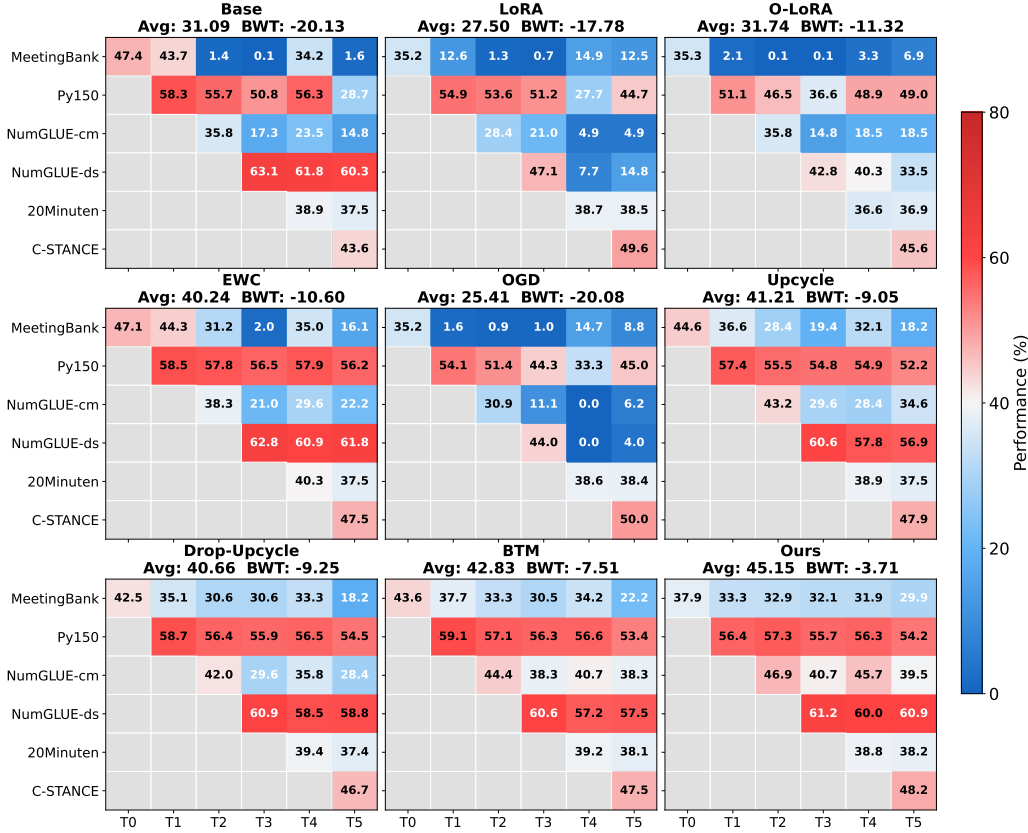


Figure 5. **Performance heatmaps (Llama-3.2-1B)**. Each cell: task (row) $\times$ stage (column); value is performance (%) after training up to that stage. From left to right, top to bottom: Base, LoRA, O-LoRA; EWC, OGD, Standard Upcycle; Drop-Upcycle, BTM, Ours.

Additional ablations and analyses covering counterfactual mask studies, CKA-NAS decoupling, hyperparameter sensitivity, joint fine-tuning upper bounds, and robustness to blurred task boundaries are deferred to Appendix F.

5.4. Ablation Studies

Necessity of each component. We perform ablations on the key components of our bilevel objective. Removing the CKA regularization term leads to noticeably higher forgetting, confirming the importance of anchoring deep-layer representations to the pre-expansion manifold. Disabling the knowledge-gain term in the outer loop causes the learned architecture to collapse to always recycling experts, showing that explicitly rewarding useful expansion is critical for achieving an effective balance between task adaptation and knowledge retention.

Robustness. Although large-scale bilevel optimization on LLMs makes exhaustive multi-seed evaluation expensive, the improvements we observe in both accuracy and BWT are substantially larger than the typical performance variability across random seeds reported on TRACE benchmarks, suggesting the robustness of our approach.

6. Conclusion

We presented our proposed method, a bilevel optimization framework for parameter-efficient MoE upcycling in continual learning. By treating expert expansion as a learnable decision via Gumbel-Softmax masks, our proposed method dynamically allocates capacity based on layer-wise representational stability measured through CKA. On the TRACE benchmark, our proposed method attains the highest Last Acc and the least-negative BWT among continual-learning baselines, while reducing the deployed model size by roughly 60% relative to standard upcycling. Our work demonstrates that upcycling an MoE model could be more efficient. This insight has significant implications for sustainable AI development, as it enables continued adaptation of large foundation models without proportional increases in computational and memory costs. Promising future directions include extending our proposed method to online continual learning settings, integrating with parameter-efficient methods like LoRA for further efficiency gains, and scaling experiments to larger models (7B+) and more diverse task sequences.

Acknowledgment

This work is supported by National Key R&D Projects (NO. 2024YFC3307100), NSFC Grant (No. 62576364), Guangdong Basic and Applied Basic Research Foundation (2026B1515020071), Shenzhen Basic Research Project (Natural Science Foundation) Basic Research Key Project (NO. JCYJ20241202124430041), Shenzhen Science and Technology Program (NO.SYSRD20250529113401002), and the China Postdoctoral Science Foundation under Grant Number 2025M781535.

Impact Statement

This paper presents our work aiming at reducing the heavy training and inference cost of cutting-edge deep learning models. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chaudhry, A., Rohrbach, M., Elhoseiny, M., Ajanthan, T., Dokania, P., Torr, P., and Ranzato, M. Continual learning with tiny episodic memories. In *Workshop on Multi-Task and Lifelong Reinforcement Learning*, 2019.
- Chen, J. C.-Y., Yun, S., Stengel-Eskin, E., Chen, T., and Bansal, M. Symbolic mixture-of-experts: Adaptive skill-based routing for heterogeneous reasoning. *arXiv preprint arXiv:2503.05641*, 2025.
- Chen, W., Zhou, Y., Du, N., Huang, Y., Laudon, J., Chen, Z., and Cui, C. Lifelong language pretraining with distribution-specialized experts. In *International Conference on Machine Learning*, pp. 5383–5395. PMLR, 2023.
- Chen, Y., Wang, S., Zhang, Y., Lin, Z., Zhang, H., Sun, W., Ding, T., and Sun, R. Mofo: Momentum-filtered optimizer for mitigating forgetting in llm fine-tuning. *arXiv preprint arXiv:2407.20999*, 2024.
- Du, W., Luo, T., Qiu, Z., Huang, Z., Shen, Y., Cheng, R., Guo, Y., and Fu, J. Stacking your transformers: A closer look at model growth for efficient llm pre-training. *Advances in Neural Information Processing Systems*, 37: 10491–10540, 2024.
- Farajtabar, M., Azizan, N., Mott, A., and Li, A. Orthogonal gradient descent for continual learning. In *International conference on artificial intelligence and statistics*, pp. 3762–3773. PMLR, 2020.
- Foret, P., Kleiner, A., Mobahi, H., and Neyshabur, B. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations*.
- Franceschi, L., Frasconi, P., Salzo, S., Grazzi, R., and Pontil, M. Bilevel programming for hyperparameter optimization and meta-learning. In *International conference on machine learning*, pp. 1568–1577. PMLR, 2018.
- French, R. M. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999.
- Gu, N., Chen, Y., Zhang, Z., Fu, P., Lin, Z., Wang, S., Sun, Y., Wu, H., Wang, W., and Wang, H. Advantageous parameter expansion training makes better large language models. *arXiv preprint arXiv:2505.24241*, 2025.

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Hu, S., Fan, Z., Shen, L., Zhang, Y., and Tao, D. Communication learning in multi-agent systems from graph modeling perspective. *IEEE Transactions on Knowledge and Data Engineering*, 2026.
- Hui, T., Zhang, Z., Wang, S., Sun, Y., Wu, H., and Su, S. Upcycling instruction tuning from dense to mixture-of-experts via parameter merging. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13012–13031, 2025.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- Komatsuzaki, A., Puigcerver, J., Lee-Thorp, J., Ruiz, C. R., Mustafa, B., Ainslie, J., Tay, Y., Dehghani, M., and Housby, N. Sparse upcycling: Training mixture-of-experts from dense checkpoints. *arXiv preprint arXiv:2212.05055*, 2022.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *International conference on machine learning*, pp. 3519–3529. PMIR, 2019.
- Li, Z. and Hoiem, D. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.
- Liao, N., Wang, X., Lin, Z., Guo, W., Hong, F., Song, S., Yu, G., Zhao, Z., Xie, S., Wei, L., et al. Innovator: Scientific continued pretraining with fine-grained moe upcycling. *arXiv preprint arXiv:2507.18671*, 2025.
- Liu, H., Simonyan, K., and Yang, Y. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.
- Mallya, A. and Lazebnik, S. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pp. 7765–7773, 2018.
- Masti, V., Liu, X., Jha, A., and Strubell, E. Stopandgrow: Accelerating large language model pre-training by growing intermediate checkpoints. 2025.
- McCloskey, M. and Cohen, N. J. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109–165. Elsevier, 1989.
- Nakamura, T., Akiba, T., Fujii, K., Oda, Y., Yokota, R., and Suzuki, J. Drop-upcycling: Training sparse mixture of experts with partial re-initialization. *arXiv preprint arXiv:2502.19261*, 2025.
- Nguyen, T., Raghu, M., and Kornblith, S. Do wide and deep networks learn the same things? uncovering how neural network representations vary with width and depth. *arXiv preprint arXiv:2010.15327*, 2020.
- Nie, Y., Han, K., Li, H., Zhou, H., Guo, T., Wu, E., Chen, X., and Wang, Y. Versatileffn: Achieving parameter efficiency in llms via adaptive wide-and-deep reuse. *arXiv preprint arXiv:2512.14531*, 2025.
- Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. Continual lifelong learning with neural networks: A review. *Neural networks*, 113:54–71, 2019.
- Qiao, F. and Mahdavi, M. Learn more, but bother less: parameter efficient continual learning. *Advances in Neural Information Processing Systems*, 37:97476–97498, 2024.
- Qiu, Z., Xu, Y., He, C., Meng, F., Xu, L., Wu, Q., and Li, H. Mingle: Mixtures of null-space gated low-rank experts for test-time continual model merging. *arXiv preprint arXiv:2505.11883*, 2025.
- Rasley, J., Rajbhandari, S., Ruwase, O., and He, Y. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 3505–3506, 2020.
- Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hadsell, R. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- Süalp, E. and Rezaei, M. Mitigating catastrophic forgetting in continual learning through model growth. *arXiv preprint arXiv:2509.01213*, 2025.

- Sukhbaatar, S., Golovneva, O., Sharma, V., Xu, H., Lin, X. V., Rozière, B., Kahn, J., Li, D., Yih, W.-t., Weston, J., et al. Branch-train-mix: Mixing expert llms into a mixture-of-experts llm. *arXiv preprint arXiv:2403.07816*, 2024.
- Team, Q. et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2(3), 2024.
- Teo, R. S. and Nguyen, T. M. Molex: Mixture of layer experts for finetuning with sparse upcycling. *arXiv preprint arXiv:2503.11144*, 2025.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Wang, R., Ding, Y., Liu, X., Wang, Y., Cheng, P., Guo, B., Zha, Z., and Gong, Y. Recycling pretrained checkpoints: Orthogonal growth of mixture-of-experts for efficient large language model pre-training. *arXiv preprint arXiv:2510.08008*, 2025.
- Wang, X., Chen, T., Ge, Q., Xia, H., Bao, R., Zheng, R., Zhang, Q., Gui, T., and Huang, X.-J. Orthogonal subspace learning for language model continual learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 10658–10671, 2023a.
- Wang, X., Zhang, Y., Chen, T., Gao, S., Jin, S., Yang, X., Xi, Z., Zheng, R., Zou, Y., Gui, T., et al. Trace: A comprehensive benchmark for continual learning in large language models. *arXiv preprint arXiv:2310.06762*, 2023b.
- Wang, Z., Yang, E., Shen, L., and Huang, H. A comprehensive survey of forgetting in deep learning beyond continual learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- Yang, E., Xin, X., Shen, L., Luo, Y., and Guo, G. Generalized embedding machines for recommender systems. *Machine Intelligence Research*, 21(3):571–584, 2024.
- Zhao, H., Wang, Z., Sun, Q., Song, K., Li, Y., Hu, X., Guo, Q., and Liu, S. Llava-cmoe: Towards continual mixture of experts for large vision-language models. *arXiv preprint arXiv:2503.21227*, 2025.
- Zhen, X., Meng, Z., Chakraborty, R., and Singh, V. On the versatile uses of partial distance correlation in deep learning. In *European Conference on Computer Vision*, pp. 327–346. Springer, 2022.

A. Limitations

The current framework has several limitations: (1) the implementation relies on the DeepSpeed framework, and we aim to refactor it into a more native PyTorch form for broader accessibility; (2) future work may extend this upcycling technique to MoE-based LLMs and multimodal LLMs; and (3) future work may adopt merging techniques to fuse the trained experts back into the original experts, eliminating extra memory burden entirely.

B. Flatness Proxy Consistency Across Checkpoints

To support the main-text claim that alternative flatness proxies induce highly consistent layer-wise rankings (Section 3), Figure 6 summarizes expert metrics across all checkpoints under the expansion regime. The panel is a 6×5 grid: rows correspond to tasks T_0 – T_5 , and columns to five complementary measures: CKA (representation similarity on old-task data), gradient norm, Fisher trace, random-direction landscape sharpness, and the top Hessian eigenvalue $\lambda_{\max}$. CKA heatmaps use a blue colormap (lighter = higher similarity); the four flatness/sensitivity proxies use a red–yellow colormap (darker = larger values, i.e., sharper or more sensitive). Across all four flatness proxies the layer-wise rankings agree closely, with deep layers consistently appearing sharper and more sensitive, which validates our choice to adopt the inexpensive gradient-norm proxy in the main algorithm.

C. Main Results on Llama3.2-3B

Table 2 reports Last Acc and BWT for Llama3.2-3B; the main text (Table 1) shows Llama3.2-1B only.

Table 2. Comparison of CL methods on Llama3.2-3B (appendix). We report Last Acc and BWT. Bold numbers denote the best method.

Metrics\Methods	Individual FT	SeqFT	LoRA	EWC	Replay	MoFO	Drop-Upcycling	BTM	Upcycling (Ours)
Last Acc	55.69	43.42 $\pm$ 0.12	36.35 $\pm$ 0.39	47.44 $\pm$ 0.42	50.66 $\pm$ 0.52	49.40 $\pm$ 0.01	47.23 $\pm$ 0.00	48.59 $\pm$ 0.00	50.96$\pm$0.00
BWT	–	-10.87 $\pm$ 0.47	-15.85 $\pm$ 0.39	-07.71 $\pm$ 0.35	-04.30 $\pm$ 0.98	-04.52 $\pm$ 0.35	-7.39 $\pm$ 0.00	-5.66 $\pm$ 0.00	-2.02$\pm$0.00

D. Complexity and Efficiency Analysis

We summarize the main trade-offs of our proposed method compared to Standard Upcycling.

Memory Footprint. During the Supernet phase, our method temporarily holds both old and new experts, resulting in a memory cost of approximately $2\times$ the expert parameters. However, this is a transient state during training only. Once the architecture is finalized, the model size is $1 + \alpha$ times the base capacity, where α is the fraction of full upcycling (e.g., $\alpha \approx 0.387$ in our experiments). In contrast, Standard Upcycling permanently imposes a $2\times$ cost. Formally, if N_E is the number of expert parameters and we denote the final expert memory of our method and Standard Upcycling by M_{ours} and M_{std} respectively, then

$$M_{\text{ours}} \approx (1 + \alpha)N_E \ll 2N_E = M_{\text{std}}. \quad (18)$$

While the peak training memory is higher than simple fine-tuning, it is in practice comparable to Standard Upcycling during the search phase, and significantly lower during inference and subsequent deployment.

Computational Cost (FLOPs). The extra cost comes from the outer-loop mask updates and CKA/flatness tracking. The mask is updated only every few inner steps, so its amortized cost is negligible, and the CKA/flatness probe runs on a small mini-batch, adding a marginal overhead relative to the standard forward–backward pass.

Table 3 compares training wall-clock, deployed size, and peak VRAM across baselines, and Table 4 reports the corresponding inference benchmark.

Table 3. Training overhead on $4\times$ A100 80GB, TRACE, Llama-3.2-1B. *Total Params* is the size of the deployed model. *Peak VRAM* is per-GPU peak during training (Supernet phase for our method, transient).

Method	Wall-clock	Total Params	Peak VRAM/GPU
SeqFT (base)	1.7h	1.24B	42.9 GB
Standard Upcycle	2.6h	6.07B	48.6 GB
Ours	2.9h	2.48B	28.8 GB
Drop-Upcycle	4.5h	6.07B	59.2 GB
BTM	4.5h	6.07B	56.4 GB

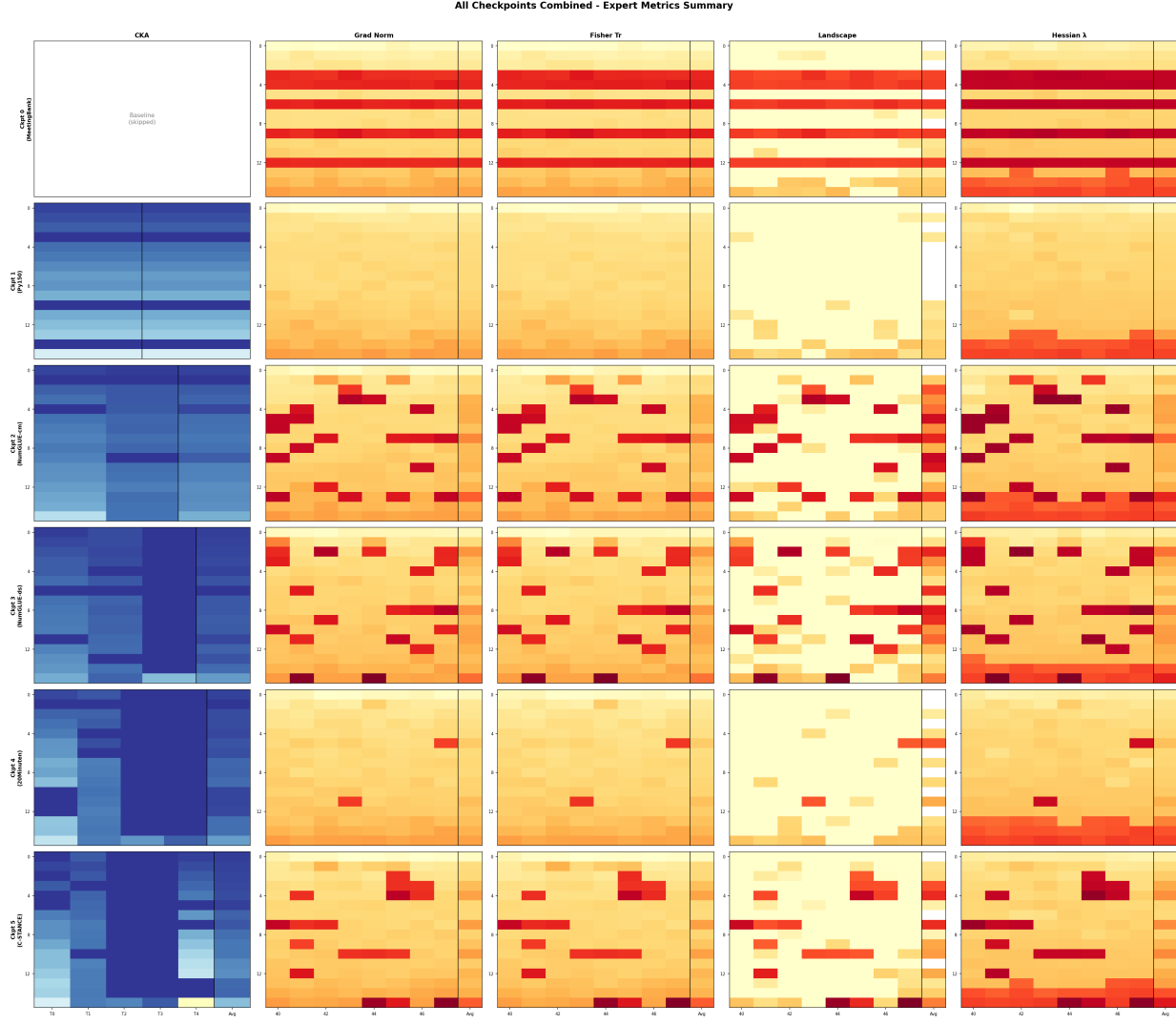


Figure 6. Expansion regime: CKA and flatness proxies across all checkpoints. Rows: tasks T_0 – T_5 ; columns: CKA, gradient norm, Fisher trace, random-direction sharpness, and $\lambda_{\max}$. The four flatness proxies induce highly consistent layer-wise rankings, supporting the main-text finding that deep layers drift more (lower CKA) and sit in sharper regions of the loss landscape, and justifying our use of the cheap gradient-norm proxy in the main algorithm.

Table 4. Inference benchmark on A100, batch = 1, seq. length = 512, FP16. *FLOPs* is per forward pass; *Latency* is per generated token.

Model	Params	FLOPs	Latency	vs Dense
Dense	1.24B	1.28T	32 ms	1.0×
Ours	2.48B	1.36T	358 ms	5.2× faster than Std
Std. Upcycle	6.07B	1.49T	1859 ms	57.6×

Convergence. We provide a simple convergence analysis for the bilevel formulation in Section G.

E. Extended Related Work

Differentiable Neural Architecture Search. Neural Architecture Search (NAS) automates the design of network architectures. DARTS (Liu et al., 2018) introduced continuous relaxation of the discrete search space, enabling gradient-based optimization. The Gumbel-Softmax estimator (Jang et al., 2016) provides a differentiable approximation to categorical sampling, making discrete architectural decisions amenable to backpropagation. Our proposed method leverages this technique to learn binary *Expand* versus *Recycle* decisions for each expert, treating architecture search as a learnable component of continual learning rather than a separate pre-training phase.

Centered Kernel Alignment. Centered Kernel Alignment (CKA) (Kornblith et al., 2019) measures the similarity between neural network representations by computing a normalized Hilbert-Schmidt Independence Criterion between activation matrices. CKA has been widely used to analyze representational similarity across layers, models, and training stages (Nguyen et al., 2020). In our work, we employ CKA as both a diagnostic tool to reveal heterogeneity in upcycling dynamics across layers and experts and as a regularization signal to prevent excessive representational drift during continual learning. From a representation perspective, this view echoes prior work that emphasizes the expressiveness of intermediate representations as the primary design variable, e.g., Generalized Embedding Machines (Yang et al., 2024) enriching embeddings via correlated-feature aggregation rather than altering the downstream interaction function.

Besides, many more model-growth works are referenced but not discussed due to page limitations.

F. Additional Ablations

This section consolidates the additional ablations and analyses referenced from Section 5. As a preliminary note, we also experimented with a layer-only CKA ablation that decides expand vs. recycle at the layer granularity (sharing one decision for all experts in a layer); this variant already improves over standard upcycling, but the full expert-wise NAS used throughout the paper provides a further gain at a lower expansion rate, motivating expert-wise decisions over layer-wise ones.

F.1. Counterfactual Mask Study

To verify that CKA guidance is causally beneficial, we run two counterfactual variants that invert our CKA-derived mask initialization, expanding stable layers and recycling drifting layers, the opposite of what the diagnostic prescribes. All runs share the effective batch size of 32 (Table 5).

Table 5. CKA counterfactual on Llama-3.2-1B (TRACE). *Invert* flips the CKA initialization but lets the bilevel learner recover; *Frozen Invert* additionally freezes the masks at the wrong assignment.

Mask Strategy	Last Acc	BWT
Ours (CKA-guided)	45.15	-3.71
Invert (learnable)	43.82	-4.75
Frozen Invert (fixed wrong masks)	44.73	-4.10

In both Last Accuracy and BWT, our CKA-guided initialization dominates the inverted counterfactuals: even when the bilevel learner is allowed to recover from a wrong initialization (*Invert, learnable*), it cannot fully close the gap. This indicates that the CKA prior is causally contributing to the final solution rather than merely correlating with it.

F.2. Hyperparameter Sensitivity

We sweep the most influential CKA weight λ_{CKA} over two orders of magnitude (Table 6). Last Acc varies within roughly $\pm 1\%$ around the default, confirming the method is not highly sensitive to this knob.

Table 6. Sensitivity of Last Acc to λ_{CKA} (single seed, final-checkpoint evaluation only).

λ_{CKA}	Last Acc
0.01	44.20
0.10 (default)	46.11
1.00	44.99

F.3. Joint Fine-Tuning Upper Bound

To contextualize the gap between sequential CL and access to all data simultaneously, we train two Joint Fine-Tuning baselines on the union of all 6 TRACE tasks (30k samples, 5 epochs, $4 \times A100$): *Joint FT (full)* trains every parameter, while *Joint FT + LoRA* restricts trainable parameters to a rank-8 LoRA on `q_proj` and `v_proj` of all 16 layers, matching the parameter budget of our CL baselines.

Table 7. Joint FT vs. our sequential approach on Llama-3.2-1B (TRACE).

Method	Last Acc	Data req.	Trainable Params
Individual FT (Table 1)	49.58	per-task model	$1.24\text{B} \times 6$
Joint FT (full)	48.36	all data simultaneously	1.24B
Joint FT + LoRA	45.18	all data simultaneously	0.85M LoRA
Ours	45.15	small replay buffer	2.48B (deployed)

Two observations: (i) Joint FT (full) is *lower* than the Individual FT upper bound, confirming multi-task interference even with simultaneous data access; (ii) under matched parameter-efficiency, our incremental method essentially ties Joint FT + LoRA (45.15 vs 45.18) while relying only on a small replay buffer rather than simultaneous access to all past data, a sequential, deployment-friendly regime.

F.4. Robustness to Task-Boundary Ambiguity: Blurry Mixed-Domain Stream

To verify that our CKA-driven expansion does not implicitly depend on oracle task labels, we construct a *Blurry Mixed-Domain Stream* that removes clean task boundaries. The 6 original TRACE tasks are reorganized into 6 equal-sized chunks (5000 samples each) with a sliding-window domain overlap, where each chunk is then internally shuffled:

Chunk	Composition
1	4000 A + 1000 B
2	1000 A + 3000 B + 1000 C
3	1000 B + 3000 C + 1000 D
4	1000 C + 3000 D + 1000 E
5	1000 D + 3000 E + 1000 F
6	1000 E + 4000 F

No chunk contains data from a single task and the expansion/recycling protocol is triggered at fixed chunk intervals, completely independent of task semantics. Evaluation is performed on the original per-task TRACE test sets for a fair comparison with the clean-boundary baseline.

Table 8. Removing task boundaries via a Blurry Mixed-Domain Stream on Llama-3.2-1B.

Setting	Last Acc	BWT	Expand Rate
Ours (clean boundaries)	45.15	-3.71	65% (157/240)
Blurry stream (no boundaries)	43.99	-1.64	74% (178/240)

Acc drops by only 1.16% ($45.15 \rightarrow 43.99$) when removing task boundaries; BWT in fact *improves* ($-3.71 \rightarrow -1.64$) because gradual domain overlap reduces catastrophic forgetting; the slightly higher expansion rate ($65\% \rightarrow 74\%$) reflects CKA correctly detecting more frequent distribution shifts in the blended data. This demonstrates that our CKA diagnostic

captures data distribution drift, not task labels: periodic evaluation on a proxy buffer suffices, matching real-world MLOps pipelines where models are updated on accumulated data batches at fixed schedules.

F.5. Bidirectional CKA↔NAS Decoupling

To verify that neither component alone delivers the result, we run two complementary single-knob ablations on Llama-3.2-1B (TRACE). *CKA only, no NAS*: keep the CKA-regularized inner loop but disable the Gumbel-Softmax mask, forcing 100% expansion. *NAS only, no CKA*: keep the outer-loop architecture search but set $\lambda_{\text{CKA}} = 0$ and initialize masks randomly. Without the CKA prior, the mask learner has no preference between deep and shallow layers and converges to near-uniform expansion, losing the parameter-efficient selectivity that motivates our framework. Only the joint diagnostic-to-architecture loop yields a sparse expansion pattern at the highest Last Acc (Table 9). The slightly better BWT of NAS-only is a trivial artifact of freezing nearly all old experts, at the cost of deployed-parameter bloat that defeats the goal of parameter-efficient MoE scaling.

Table 9. Bidirectional ablation of the CKA↔NAS loop on Llama-3.2-1B (TRACE).

Variant	Last Acc	BWT	Expand Rate
CKA only, no NAS	44.20	-4.05	100% (240/240)
NAS only, no CKA	44.79	-2.85	96% (230/240)
Ours (CKA + NAS)	45.15	-3.71	65% (157/240)

G. Convergence of the Outer-Loop (Mask) Update

Inspired by (Hu et al., 2026), we provide a convergence guarantee for the outer-loop (mask) update when the gradient is estimated via Gumbel-Softmax. The outer variable is the mask logit α ; we denote the outer objective $\mathcal{L}_{\text{mask}}(\alpha)$ and the learning rate for the mask update by η . We use the following assumptions and then state the theorem in the paper’s notation.

Assumption G.1 (Unbiased gradient). Let $\mathbf{g}^t$ be the gradient of $\mathcal{L}_{\text{mask}}$ with respect to α estimated by Gumbel-Softmax at step t . Then $\mathbb{E}[\mathbf{g}^t] = \nabla_{\alpha} \mathcal{L}_{\text{mask}}(\alpha^t)$.

Assumption G.2 (Bounded variance). Let $\mathbf{g}^t$ be as in Assumption G.1. There exists a constant $\sigma^2 \geq 0$ such that $\mathbb{E}[\|\mathbf{g}^t - \nabla_{\alpha} \mathcal{L}_{\text{mask}}(\alpha^t)\|^2] \leq \sigma^2$.

Assumption G.3 (Smoothness). The function $\mathcal{L}_{\text{mask}} : \mathbb{R}^d \rightarrow \mathbb{R}$ is B -smooth: for all $\mathbf{u}, \mathbf{v}$,

$$\mathcal{L}_{\text{mask}}(\mathbf{v}) \leq \mathcal{L}_{\text{mask}}(\mathbf{u}) + \langle \nabla \mathcal{L}_{\text{mask}}(\mathbf{u}), \mathbf{v} - \mathbf{u} \rangle + \frac{B}{2} \|\mathbf{v} - \mathbf{u}\|^2. \quad (19)$$

Theorem G.4 (Convergence of average gradient norm). Suppose $\mathcal{L}_{\text{mask}}$ satisfies Assumptions G.1, G.2, and G.3. Consider the outer-loop update $\alpha^{t+1} = \alpha^t - \eta \mathbf{g}^t$ with learning rate η such that $0 < \eta < 2/B$. Then the average expected squared gradient norm over T steps satisfies

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla_{\alpha} \mathcal{L}_{\text{mask}}(\alpha^t)\|^2] \leq \frac{C_1}{T} + C_2, \quad (20)$$

where $C_1 = \frac{\mathcal{L}_{\text{mask}}(\alpha^0) - \mathcal{L}_{\text{mask}}^*}{\eta - B\eta^2/2}$ and $C_2 = \frac{B\eta^2\sigma^2}{2(\eta - B\eta^2/2)}$, and $\mathcal{L}_{\text{mask}}^*$ denotes a lower bound of $\mathcal{L}_{\text{mask}}$. In particular, as $T \rightarrow \infty$, the average expected squared gradient norm is bounded by C_2 .

Proof. By B -smoothness of $\mathcal{L}_{\text{mask}}$ (Assumption G.3),

$$\begin{aligned} \mathcal{L}_{\text{mask}}(\alpha^{t+1}) &\leq \mathcal{L}_{\text{mask}}(\alpha^t) + \langle \nabla \mathcal{L}_{\text{mask}}(\alpha^t), \alpha^{t+1} - \alpha^t \rangle + \frac{B}{2} \|\alpha^{t+1} - \alpha^t\|^2 \\ &= \mathcal{L}_{\text{mask}}(\alpha^t) - \eta \langle \nabla \mathcal{L}_{\text{mask}}(\alpha^t), \mathbf{g}^t \rangle + \frac{B\eta^2}{2} \|\mathbf{g}^t\|^2. \end{aligned} \quad (21)$$

Let $\delta^t = \mathbf{g}^t - \nabla \mathcal{L}_{\text{mask}}(\alpha^t)$. Then $\mathbf{g}^t = \nabla \mathcal{L}_{\text{mask}}(\alpha^t) + \delta^t$. Substituting and expanding,

$$\begin{aligned} \mathcal{L}_{\text{mask}}(\alpha^{t+1}) &\leq \mathcal{L}_{\text{mask}}(\alpha^t) - \eta \|\nabla \mathcal{L}_{\text{mask}}(\alpha^t)\|^2 - \eta \langle \nabla \mathcal{L}_{\text{mask}}(\alpha^t), \delta^t \rangle \\ &\quad + \frac{B\eta^2}{2} \|\nabla \mathcal{L}_{\text{mask}}(\alpha^t) + \delta^t\|^2. \end{aligned} \quad (22)$$

Using $\|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t) + \boldsymbol{\delta}^t\|^2 = \|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)\|^2 + 2\langle \nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t), \boldsymbol{\delta}^t \rangle + \|\boldsymbol{\delta}^t\|^2$ and rearranging,

$$\mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^{t+1}) \leq \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t) - (\eta - B\eta^2)\|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)\|^2 + (-\eta + B\eta^2)\langle \nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t), \boldsymbol{\delta}^t \rangle + \frac{B\eta^2}{2}\|\boldsymbol{\delta}^t\|^2. \quad (23)$$

Taking expectation, and using $\mathbb{E}[\boldsymbol{\delta}^t] = \mathbf{0}$ (Assumption G.1) and $\mathbb{E}[\|\boldsymbol{\delta}^t\|^2] \leq \sigma^2$ (Assumption G.2), we have $\mathbb{E}[\|\mathbf{g}^t\|^2] \leq \mathbb{E}[\|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)\|^2] + \sigma^2$. Hence

$$\mathbb{E}[\mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^{t+1})] \leq \mathbb{E}[\mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)] - \left(\eta - \frac{B\eta^2}{2}\right)\mathbb{E}[\|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)\|^2] + \frac{B\eta^2}{2}\sigma^2. \quad (24)$$

Rearranging and summing over $t = 0, 1, \dots, T-1$,

$$\left(\eta - \frac{B\eta^2}{2}\right) \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)\|^2] \leq \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^0) - \mathbb{E}[\mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^T)] + T \cdot \frac{B\eta^2}{2}\sigma^2. \quad (25)$$

Bounding $\mathbb{E}[\mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^T)] \geq \mathcal{L}_{\text{mask}}^*$ and dividing by $T(\eta - B\eta^2/2)$,

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla \mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^t)\|^2] \leq \frac{C_1}{T} + C_2. \quad (26)$$

Hence as $T \rightarrow \infty$, the average expected squared gradient norm is bounded by C_2 . $\square$

Remark G.5. The learning rate condition $\eta < 2/B$ ensures $\eta - B\eta^2/2 > 0$. The constants C_1 and C_2 depend on the initial gap $\mathcal{L}_{\text{mask}}(\boldsymbol{\alpha}^0) - \mathcal{L}_{\text{mask}}^*$, the variance σ^2 of the Gumbel-Softmax gradient estimator, and the smoothness B . Smaller η reduces C_2 but increases C_1/T ; a typical choice is $\eta = O(1/\sqrt{T})$ for a decaying schedule.

H. Hyperparameters

Table 10. Hyperparameters for Bilevel Optimization

Parameter	Symbol	Default	Description
<i>Temperature Annealing</i>			
Initial temperature	τ_0	5.0	Soft decisions at start
Final temperature	τ_{final}	0.1	Hard decisions at end
Decay rate	ρ	0.995	Per mask-update decay
<i>Balanced Loss</i>			
Knowledge gain weight	γ	0.25	Weight for expansion signal
Target expand rate	r_{target}	0.35	Default expansion target
Task 1 expand rate	r_{task1}	1.0	100% for first supernet task
Balance weight	β	0.5	Under-expansion penalty
Sparsity weight	σ	0.05	Over-expansion penalty
<i>Optimization</i>			
Mask update interval	N	10	Steps between mask updates
Mask learning rate	η_π	0.01	Adam LR for mask learner
Replay buffer size	B	200	Batches per task
<i>CKA Regularization</i>			
CKA weight	λ_{CKA}	0.15	Base CKA regularization
Replay weight	λ_{replay}	0.8	Replay loss weight
CKA layers	-	deep	Last 6 layers